

Grid

Selection Strategies

We implemented two selection strategies combined with elitism in `generate_successors()`.

- First is tournament selection, we used $k=10$, in which we pick 10 random individuals from the population and return the best. Being selective while remaining diverse.
- Second is the roulette wheel selection, in which each individual's selection probability is proportional to its fitness. This gives weaker individuals a chance to reproduce, maintaining population diversity, while still able to improve fitness. It is used as an alternative to tournament selection (it is implemented but not used for the final map selection as we found out that the increasing rate of fitness is slower with this selection comparing to tournament selection)

Elitism is which the top individuals are carried forward unchanged into the next generation. Guaranteeing that the best solution found so far is always in the population for crossover or mutation.

Crossover

We used a single-point vertical crossover operating on columns. A random column position is selected as the crossover point. Child 1 received parent A's columns to the left of the point and parent B's columns to the right. Child 2 receives the inverse, creating two children per pair. We use column-based crossover because we do not want to break the sections of pipelines, walls, etc. Cutting by columns preserves these structures on each side while combining different level sections from each parent.

Before deciding to use single-point crossover, we tried uniform vertical crossover (each line is randomly selected to be from parent A or parent B) and decided not to proceed because that will break the horizontal structures such as the staircase.

Mutation

For the mutation section, we added per-tile mutations, structural mutations, and some constraints to prevent weird generation.

For per-tile mutations, each selected tile is replaced with a randomly chosen tile type. The selection is weighted heavily toward empty space using the weights [120, 10, 4, 2, 3, 6, 4], and the starting area and the ending area are protected from mutation to preserve solvability.

We also added structural mutation, with a 3% chance of creating a ground gap with a width of two to four tiles, and have the system create upward or downward staircases when an X block appears near an existing wall.

To keep the level valid, we added a list of constraints after mutations, including

- ground gaps limited to a maximum width of 4 tiles to remain jumpable
- walls and pipes above gaps are removed because they must be supported by the ground floating X blocks are corrected. If an X structure is four tiles high or shorter, it is extended downward to form a wall. It is also filled to the ground when it connects with a nearby wall to form a staircase. Otherwise, unsupported X blocks are removed.

- Vertical stacks of B blocks are broken apart because B blocks are intended to act as floating platforms rather than wall materials.
- M and ? blocks must have a solid tile within four spaces below them so the player can reach them by jumping.
- Coins must have a solid tile within two spaces below for the same reason.
- The space below ?, B, and M blocks must also remain empty so the player can hit them from underneath.
- The top four rows of the level are always cleared

Fitness Calculation

We kept the original weighted fitness metrics and added several new rewards to encourage more interesting levels. The decoration percentage provides a scaled bonus of up to +0.5, rewarding levels that contain visual variety without becoming overcrowded. Meaningful jumps with a bonus of up to +0.5, rewarding jumps that require actual platforming rather than simple movement across flat ground. The total number of jumps can also add up to +0.3, encouraging levels that include some vertical movement and do not feel like flat walking sections. We also added a leniency penalty, reducing the fitness score when a level is either too easy or too difficult. The goal is to guide the genetic algorithm toward levels that are challenging but still fair and playable.

Random Population Initialization

The initial population is generated using random.choices with weights strongly favoring empty space to prevent the randomly generated levels from starting out overly cluttered.

After each level is generated, the same validity constraints used during mutation are applied. These checks fix unsupported enemies, unreachable items, floating walls, and other structural problems. , Invalid pipes are also checked, pipe tops are filled with pipe segments when they are close enough to the ground, and X blocks within four rows of the ground are extended downward to form walls.

We've also thought about implementing a structural initialization approach, meaning to build levels from scratch by placing a set number of platforms, pipes, staircases, and enemies at valid positions, however we decided to not proceed with the approach because that would reduce the initial diversity of the population.

DE

Unlike the Grid method, where every tile is stored directly in the genome, the Design Element (DE) method stores a list of higher-level level components, and each gene represents a complete design element (i.e. platforms, pipes with head and body, holes, and stairs). The random_individual() function generates a random list of these design elements with the random parameters to construct each of them.

The `to_level()` function first creates an empty base level with only the ground, Mario's starting position, and the goal flag. It then iterates through every design element and draws each one onto the map according to its parameters. For example, a pipe design element automatically generates both the pipe head and all of its pipe segments, while a platform element generates a continuous horizontal platform of the stated width and material.

The crossover method used in DE independently selects one cut point in each parent's design element list. The first child (ga) takes the left part of the first parent's genome and the right part of the second parent's genome. The second child (gb) does the opposite, taking the left part of the second parent's genome and the right part of the first parent's genome. Since each gene represents a complete design element, such as a pipe or platform, the scope of the crossover is at the element/module level instead of the individual grid tile, which allows complete level structures to be inherited by the offspring (e.g. the stairs or platform won't be splitted into half by the crossover).

The DE mutate function first gives each child a 10% chance of mutation. The mutation here randomly selects one design element instead of a tile. And based on the element type, it changes one of that element's parameters to modify element's position, width, height, or material. (e.g. a pipe may change its x position or height; a platform may change its x-y position, width, or material). The original design element is then removed and replaced with the modified version. Similar to crossover, since mutate function modifies design elements by adjusting their parameters, their overall structure can be preserved.

DE Fitness Function Improvement

We improved the DE fitness function by evaluating both the rendered level and the composition of its design-element genome. The original fitness score was calculated from existing metrics such as solvability, path percentage, empty space, linearity, and jump variance. We kept these measurements, but added penalties based on the number of certain design elements.

The function first counts how many elements of each type appear in the genome. It then penalizes excessive stairs and holes, since too many of these elements can make a level visually repetitive or difficult to navigate. The penalty increases based on how far the element count exceeds the allowed limit. We also penalize levels with no platforms to encourage more varied structures. In addition, bonuses are awarded for having a moderate number of coins and question blocks, since these elements can make the level more rewarding and visually interesting. Upper limits are used so that the algorithm does not simply generate excessive amounts of these elements to increase fitness. The final fitness score combines the original metric-based score with these penalties and bonuses. Our goal was to let the fitness function consider the balance and variety of the design elements when constructing the level.

Other Modifications